МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”
КАФЕДРА ИНТЕЛЛЕКТУАЛЬНЫХ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ

ОТЧЁТ

по лабораторной работе №2

Выполнил:
студент группы ПО-12
Снарский П.П.
Проверила:
Кулик А.Д.

Брест 2026

Цель работы: закрепить навыки объектно-ориентированного программирования на языке Python

Вариант 12

Ход работы

Задание 1. Реализовать простой класс

Требования к выполнению

• Реализовать пользовательский класс по варианту.

Для каждого класса

• Создать атрибуты (поля) классов

• Создать методы классов

• Добавить необходимые свойства и сеттеры (по необходимости)

• Переопределить магические методы __str__ и __eq__

Равносторонний треугольник, заданный длинами сторон – Предусмотреть возможность определения площади и периметра, а также логический метод, определяющий существует или такой треугольник. Конструктор должен позволять создавать объекты с начальной инициализацией. Переопределить метод __eq__, выполняющий сравнение объектов данного типа.

Код программы:

```
"""
Make class of triangle
"""


import math


class Triangle:
    """Class Triangle"""
    def __init__(self, a=5, b=5, c=5):
        self.a = a
        self.b = b
        self.c = c
        self.perimeter = self.a + self.b + self.c
        self.area = math.sqrt(
            self.perimeter / 2
            * (self.perimeter / 2 - self.a)
            * (self.perimeter / 2 - self.b)
            * (self.perimeter / 2 - self.c)
        )
        self.is_exist = (self.perimeter - max(a, b, c)) > max(a, b, c)
        self.is_equilateral = a == b == c

    def __eq__(self, other):
        sides1 = sorted([self.a, self.b, self.c])
        sides2 = sorted([other.a, other.b, other.c])
```

```
        return sides1 == sides2

    def __str__(self):
        return (
            f"Sides: {self.a}, {self.b}, {self.c}\nExists?: {self.is_exist}\n"
            f"Perimeter: {self.perimeter}\n"
            f"Area: {self.area}\n Equilateral?: {self.is_equilateral}\n"
        )


def main():
    """Main function"""
    triangle_1 = Triangle(
        int(input("Enter side a ")), int(input("Enter side b ")), int(input("Enter side
c "))
    )
    triangle_2 = Triangle(
        int(input("Enter side a ")), int(input("Enter side b ")), int(input("Enter side
c "))
    )
    print(triangle_1)
    print(triangle_2)
    print(triangle_1 == triangle_2)


main()
```

Задание 2. Построить модель программной системы с применением отношений (обобщения, агрегации, ассоциации, реализации) между классами. Задать атрибуты и методы классов. Реализовать (если необходимо) дополнительные классы. Продемонстрировать работу разработанной системы. 1) Система Факультатив. Преподаватель объявляет запись на Курс. Студент записывается на Курс, обучается и по окончании Преподаватель выставляет Оценку, которая сохраняется в Архиве. Студентов, Преподавателей и Курсов при обучении может быть несколько.

Код программы:

```
"""Making Course system"""


class Course:
    """Course class"""

    def __init__(self, name, teacher):
        self.name = name
        self.students = []
        self.teacher = teacher

    def add_student(self, student):
        """Add a student to the class"""

        self.students.append(student)

    def conduct_course(self):
        """Conducting course"""

        print(f"\nCourse '{self.name}' started. Teacher: {self.teacher.name}")
        for student in self.students:
            print(f"{student.name} is studying...")
```

```python
class Teacher():
    """Teacher class"""

    def __init__(self, name):
        self.name = name

    def assign_mark(self, student, course, value):
        """Assign mark to student"""

        mark = Mark(student, course, value)
        Archive.add_record(mark)
        print(f"{self.name} gave mark {mark.value} to {student.name} for course
{course.name}")

    def show_teacher(self):
        """Showing teacher"""

        print(f"\nTeacher {self.name} is teaching...")


class Student():
    """Student class"""

    def __init__(self, name):
        self.name = name
        self.courses = []

    def enroll(self, course):
        """Enroll student with course"""

        self.courses.append(course)
        course.add_student(self)
        print(f"{self.name} enrolled on {course.name}")

    def show_student(self):
        """Showing student"""
        print(f"\nStudent {self.name} is enrolled...")


class Mark:
    """Mark class"""

    def __init__(self, student, course, value):
        self.student = student
        self.course = course
        self.value = value

    def __str__(self):
        return f"{self.student.name} | {self.course.name} | {self.value}"

    def show_mark(self):
        """Showing mark"""
        print(self)


class Archive:
    """Archive class"""

    records = []

    @classmethod
    def add_record(cls, mark):
        """Adding record"""

        cls.records.append(mark)

    @classmethod
```

```python
    def show_archive(cls):
        """Showing archive"""

        print("\n=== Archive ===")
        for mark in cls.records:
            print(mark)


def main():
    """Main function"""

    teacher1 = Teacher("Rogers")
    teacher2 = Teacher("Stark")

    course1 = Course("Course1", teacher1)
    course2 = Course("Course2", teacher2)

    s1 = Student("Bucky")
    s2 = Student("Sam")
    s3 = Student("Peter")

    s1.enroll(course1)
    s2.enroll(course1)
    s3.enroll(course2)

    course1.conduct_course()
    course2.conduct_course()

    teacher1.assign_mark(s1, course1, 9)
    teacher1.assign_mark(s2, course1, 10)
    teacher2.assign_mark(s3, course2, 10)

    Archive.show_archive()


main()
```

Вывод: в ходе выполнения лабораторной работы закрепи навыки объектно-ориентированного программирования на языке Python